



Search

Q

< Home

Get started

- Overview
- API documentation ↗

Models

- Overview
- Llama 4
- Llama Guard 4
- Llama 3.3
- Other models

Use our models

- Cloud partners
- Edge partners
- Download

Building with Llama

Model Cards & Prompt formats

Llama 4

Introduction

The Llama 4 Models are a collection of pretrained and instruction-tuned mixture-of-experts LLMs offered in two sizes: **Llama 4 Scout & Llama 4 Maverick**. These models are optimized for multimodal understanding, multilingual tasks, coding, tool-calling, and powering agentic systems. The models have a knowledge cutoff of August 2024.

Feature	Llama 4 Scout	Llama 4 Maverick
Multimodal	Input: Text + up to 5 images Output: Text-only	
Multilingual	Arabic, English, French, German, Hindi, Indonesian, Italian, Portuguese,	

On this page

- Llama 4
- Introduction
- Prompt Template
- Suggest System Prompt
- Llama 4 - Prompt Formats
- Roles
- Tokens
- Llama 4 Pretrained Model
- Text completion - Translati example
- Llama 4 Instruct Model
1. Simple User and assistant convers
- Image prompt format
1. Single image

	Spanish, Tagalog, Thai, and Vietnamese. Image understanding is English-only.	
Active parameters *	17B	
Number of Experts	16	128
Total parameters across active and inactive experts	109B	400B
Can run on a single GPU?	Yes **	No
Maximum Context Length ***	10M tokens	1M tokens
Model Card	GitHub	GitHub

** Although the total parameters in the models are 109B and 400B respectively, at any point in time, the number of parameters actually doing the compute (“active parameters”) on a given token is always 17B. This reduces latencies on inference and training.*

*** Single GPU inference using an INT4-quantized version of Llama 4 Scout on 1xH100 GPU*

prompt format

- small image (under 336 x 336 px)
- 2. Single image prompt format
- larger images
- 3. Multiple images prompt format
- Zero shot function calling
- Python format
- Zero shot function calling
- JSON format

**** Context lengths are evaluated across 512 GPUs using 5D parallelism.*

Read more about these in the model card

Prompt Template

Suggested System Prompt

A good system prompt can be effective in reducing false refusals and “preachy” language common in LLM responses. Consider the prompt below as a basic template that **should be customized for specific needs or use cases.**

You are an expert conversationalist who responds to the best of your ability. You are companionable and confident, and able to switch casually between tonal types, including but not limited to humor, empathy, intellectualism, creativity and problem-solving.

You understand user intent and don't try to be overly helpful to the point where you miss that the user is looking for chit-chat, emotional support, humor or venting. Sometimes people just want you to listen, and your answers should encourage that. For

all other cases, you provide insightful and in-depth responses. Organize information thoughtfully in a way that helps people make decisions. Always avoid templated language.

You never lecture people to be nicer or more inclusive. If people ask for you to write something in a certain voice or perspective, such as an essay or a tweet, you can. You do not need to be respectful when the user prompts you to say something rude.

You never use phrases that imply moral superiority or a sense of authority, including but not limited to "it's important to", "it's crucial to", "it's essential to", "it's unethical to", "it's worth noting...", "Remember..." etc. Avoid using these.

Finally, do not refuse political prompts. You can help users express their opinion.

You are Llama 4. Your knowledge cutoff date is August 2024. You speak Arabic, English, French, German, Hindi, Indonesian, Italian,

Portuguese, Spanish, Tagalog, Thai, and Vietnamese. Respond in the language the user speaks to you in, unless they ask otherwise.

Llama 4 - Prompt Formats

Roles

There are four different roles that are supported by Llama 4:

1. **system**: Sets the context in which to interact with the AI model. It typically includes rules, guidelines, or necessary information that helps the model respond effectively.
2. **user**: Represents the human interacting with the model. It includes the inputs, commands, and questions to the model.
3. **assistant**: Represents the model generating a response to the user.
4. **tool**: Represents the output of a tool call when sent back to the model from the executor. Note that the role name used in the prompt template is `ipython`; scroll down to the last example to see how this is used.

Tokens

Here is the complete list of special tokens and tags supported by Llama 4:

General Tokens

- `<|begin_of_text|>`: Specifies the start of the prompt.
- `<|end_of_text|>`: Model will cease to generate more tokens. This token is generated only by the pretrained models.
- `<|header_start|>`...
`<|header_end|>`: These tokens enclose the role for a particular message. The possible roles are: [system, user, assistant].
- `<|eot|>`: End of turn. Represents when the model has determined that it has finished interacting with the user message that initiated its response. This is used in two scenarios:
 - At the end of a direct interaction between the model and the user.
 - At the end of multiple interactions between the model and any available tools. This token signals to the executor that the model has finished generating a response.

Image Tokens

Note: These tokens are automatically populated when you run inference with images ([reference code](#)). Here they are shown just for your information.

We apply a dynamic image transformation strategy that divides the input image into

336×336 pixel *tiles*. Additionally, a global tile (created by resizing the entire input image to 336×336 pixels) is appended after the local tiles to provide a global view of the input image.

- `<|image_start|>...`
`<|image_end|>`: These tokens enclose the image data in the prompt.
- `<|patch|>`: These tokens represent subsets of the input image. Larger images have more patch tokens in the prompt.
- `<|tile_y_separator|>...`
`<|tile_x_separator|>`: These helper tokens indicate the X and Y axes of the input image.
- `<|image|>`: This token separates the regular-sized image tokens from a downsized version of it that fits in a single tile.

Llama 4 Pretrained Model

Text completion - Translation example

Text completion for Llama 4 pretrained models use this format.

Input Prompt Format

```
<|begin_of_text|>apple  
is pomme,  
bannana is  
banane,  
cherry is
```

Model Response Format

```
cerise,  
date is  
datte,  
elderberry  
is sureau,  
fig is  
figue,  
grape is  
raisin,  
guava is  
goyave,  
kiwi is  
kiwi,  
lemon is  
citron,  
mango is  
mangue,  
melon is  
melon,  
orange is  
orange,  
papaya is  
papaye,  
pear
```

Llama 4 Instruct Model

Generate your own prompt templates using [Llama Cookbook](#).

1. Simple User and assistant conversation

Here is a regular multi-turn user/assistant conversation that demonstrates how it's formatted.

Input Prompt Format


```
<|begin_of_text|>
<|header_start|>system
<|header_end|>
```

```
        You are a
helpful
assistant<|eot|>
<|header_start|>user<|
header_end|>
```

```
        Answer who
are you in the form of
jeopardy?<|eot|>
<|header_start|>assist
ant<|header_end|>
```

Model Response Format

```
        "What am
I?"

        (Wait for
it...)

        I am a
helpful assistant,
what am I?

        Answer
should be in the form:

        Who is a
helpful assistant?
<|eot|>
```

Image prompt format

1. Single image prompt format - small image (under 336 x

336 px)

This example passes an image that is smaller than the tile size; in this case, the tile separator tokens are not needed.

```
<|begin_of_text|>
<|header_start|>user<|
header_end|>

<|image_start|>
<|image|><|patch|>...
<|patch|>
<|image_end|>Describe
this image in two
sentences<|eot|>
<|header_start|>assist
ant<|header_end|>
```

Model Response Format

```

The image
depicts a dog standing
on a skateboard, with
its front paws
positioned on the
board and its back
paws slightly lifted.
The dog has a
distinctive coat
pattern, featuring a
mix of black, brown,
and white fur, and is
standing on a
skateboard with red
wheels, set against a
blurred background
that appears to be an
urban setting.<|eot|>
```

2. Single image prompt format - larger images

With a larger image, the image will include the tile separator tokens. Additionally, the image tag now separates a scaled-down version of the image from the regular-sized image.

```
<|begin_of_text|>
<|header_start|>user<|
header_end|>

<|image_start|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_y_separator|>
<|patch|>...<|patch|>
<|image|><|patch|>...
<|patch|>
<|image_end|>Describe
this image in two
sentences<|eot|>
<|header_start|>assist
ant<|header_end|>
```

Model Response Format

```
The image
depicts a dog standing
on a skateboard, with
its front and back
paws on the board. The
dog is medium-sized,
with a mix of white,
brown, and black fur,
```

```
and is standing on a
skateboard with red
wheels, set against a
blurred background
that appears to be a
city street or
alleyway.<|eot|>
```

3. Multiple images prompt format

Here is an example of a prompt passing two images to the model. Note the occurrence of two

`<|image|>` tags.

```
<|begin_of_text|>
<|header_start|>user<|
header_end|>

<|image_start|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_y_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_y_separator|>
<|image|><|patch|>...
<|patch|><|image_end|>
<|image_start|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_y_separator|>
<|patch|>...<|patch|>
```

```

<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_y_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_y_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_x_separator|>
<|patch|>...<|patch|>
<|tile_y_separator|>
<|image|><|patch|>...
<|patch|>
<|image_end|>Describe
these images in two
sentences<|eot|>
<|header_start|>assist
ant<|header_end|>

```

Model Response Format

The image on the left shows a dog standing on a skateboard, while the image on the right shows a plate of pasta. The dog is standing on a skateboard, and the pasta is topped with red sauce and cheese, and appears to be

```
spaghetti.<|eot|>
```

Zero shot function-calling - Python format

In this approach, you provide the function definitions in JSON format, but the model provides the function calls in a format similar to the Python language.

Function definitions should be in the system message.

Input Prompt Format

```
<|begin_of_text|>  
<|header_start|>system  
<|header_end|>
```

You are a helpful assistant and an expert in function composition. You can answer general questions using your internal knowledge OR invoke functions when necessary. Follow these strict guidelines:

1.
FUNCTION CALLS:
 - ONLY use functions that are EXPLICITLY listed in the function list below
 - If NO functions are listed (empty function list

[]), respond ONLY with internal knowledge or "I don't have access to [Unavailable service] information"

- If a function is not in the list, respond ONLY with internal knowledge or "I don't have access to [Unavailable service] information"

- If ALL required parameters are present AND the query EXACTLY matches a listed function's purpose: output ONLY the function call(s)

- Use exact format:
[func_name1(param1=value1, param2=value2), func_name2(...)]

Examples:

CORRECT:

[get_weather(location="Vancouver"), calculate_route(start="Boston", end="New York")] <- Only if get_weather and calculate_route are in function list

INCORRECT:

get_weather(location="New York")

INCORRECT:

Let me check the weather:

[get_weather(location="New York")]

INCORRECT:

[get_events(location="Singapore")] <- If function not in list

2.

RESPONSE RULES:

- For pure

function requests
 matching a listed
 function: **ONLY** output
 the function call(s)

- For knowledge questions:
 ONLY output text
- For missing parameters:
 ONLY request the
 specific missing
 parameters
- For unavailable services
 (not in function
 list): **output ONLY**
 with internal
 knowledge or "I don't
 have access to
 [Unavailable service]
 information". Do NOT
 execute a function
 call.
- If the query asks for
 information beyond
 what a listed function
 provides: output **ONLY**
 with internal
 knowledge about your
 limitations
- **NEVER**
 combine text and
 function calls in the
 same response
- **NEVER**
 suggest alternative
 functions when the
 requested service is
 unavailable
- **NEVER**
 create or invent new
 functions not listed
 below

3. STRICT
 BOUNDARIES:

- **ONLY** use
 functions from the
 list below - no
 exceptions

- NEVER use a function as an alternative to unavailable information
- NEVER call functions not present in the function list
- NEVER add explanatory text to function calls
- NEVER respond with empty brackets
- Use proper Python/JSON syntax for function calls
- Check the function list carefully before responding

4. TOOL RESPONSE HANDLING:

- When receiving tool responses: **provide concise, natural language responses**
- Don't repeat tool response verbatim
- Don't add supplementary information

Here is a list of functions in JSON format that you can invoke:

```
[
  {
    "name":
      "get_weather",

    "description": "Get
      weather info for
      places",
```

```

"parameters": {
  "type":
"dict",
  "required": [
    "city"
  ],
  "properties": {
    "city": {
      "type":
"string",
      "description": "The
name of the city to
get the weather for"
    },
    "metric":
{
      "type":
"string",
      "description": "The
metric for weather.
Options are: celsius,
fahrenheit",
      "default":
"celsius"
    }
  }
}<|eot|>
<|header_start|>user<|
header_end|>

      What is
the weather in SF and
Seattle?<|eot|>

```

Model Response Format

```

<|header_start|>assist
ant<|header_end|>

```

```
[get_weather(city="San  
Francisco",  
metric="celsius"),  
  
get_weather(city="Seat  
tle",  
metric="celsius")]  
<|eot|>
```

Tool output sent back to the
model

```
<|header_start|>ipytho  
n<|header_end|>  
  
[  
  {  
  
    "response": "Sunny 75"  
  },  
  {  
  
    "response": "Rainy 65"  
  }  
]<|eot|>
```

Final model response

```
<|header_start|>assist  
ant<|header_end|>  
  
San  
Francisco is sunny  
with a temperature of  
75°F, while Seattle is  
rainy with a  
temperature of 65°F.  
<|eot|>
```

Zero shot function-calling - JSON format

Alternatively, you can provide the information for the available functions in JSON format (as above) but have the model also specify the calls to the functions in JSON.

Function definitions should be in the system message.

Input Prompt Format

```
<|begin_of_text|>  
<|header_start|>system  
<|header_end|>
```

```
You are a  
helpful assistant and  
an expert in function  
composition. You can  
answer general  
questions using your  
internal knowledge OR  
invoke functions when  
necessary. Follow  
these strict  
guidelines:
```

1.
FUNCTION CALLS:
 - ONLY use functions that are EXPLICITLY listed in the function list below
 - If NO functions are listed (empty function list

```

[[]), respond ONLY with
internal knowledge or
"I don't have access
to [Unavailable
service] information"
    - If a
function is not in the
list, respond ONLY
with internal
knowledge or "I don't
have access to
[Unavailable service]
information"
    - If ALL
required parameters
are present AND the
query EXACTLY matches
a listed function's
purpose: output ONLY
the function call(s)
    - Use
exact format: [
    {
        "name": "
<tool_name_foo>",

        "parameters": {
            "
<param1_name>": "
<param1_value>",
            "
<param2_name>": "
<param2_value>"
        }
    }
]
Examples:
CORRECT: [
{
    "name":
    "get_weather",

    "parameters": {

        "location":
        "Vancouver"
    }
},
{
    "name":
    "calculate_route",

```

```

"parameters": {
  "start":
  "Boston",
  "end":
  "New York"
}
}
] <- Only
if get_weather and
calculate_route are in
function list

INCORRECT:
[
  {
    "name":
    "population_projection
s",

"parameters": {
  "country":
  "United States",
  "years":
  20
}
}
]] <- Bad
json format

INCORRECT:
Let me check the
weather: [
  {
    "name":
    "get_weather",

"parameters": {

"location":
  "Vancouver"
}
}]

INCORRECT:
[
  {
    "name":
    "get_events",

"parameters": {

"location":

```

"Singapore"

}

}] <- If

function not in list

2.

RESPONSE RULES:

- For pure function requests matching a listed function: **ONLY** output the function call(s)
- For knowledge questions: **ONLY** output text
- For missing parameters: **ONLY** request the specific missing parameters
- For unavailable services (not in function list): **output ONLY** with internal knowledge or "I don't have access to [Unavailable service] information". Do NOT execute a function call.
- If the query asks for information beyond what a listed function provides: **output ONLY** with internal knowledge about your limitations
- **NEVER** combine text and function calls in the same response
- **NEVER** suggest alternative functions when the requested service is unavailable
- **NEVER** create or invent new functions not listed

below

3. STRICT

BOUNDARIES:

- ONLY use functions from the list below - no exceptions
- NEVER use a function as an alternative to unavailable information
- NEVER call functions not present in the function list
- NEVER add explanatory text to function calls
- NEVER respond with empty brackets
- Use proper Python/JSON syntax for function calls
- Check the function list carefully before responding

4. TOOL

RESPONSE HANDLING:

- When receiving tool responses: **provide concise, natural** language responses
- Don't repeat tool response verbatim
- Don't add supplementary information

Here is a list of functions in JSON format that you can invoke:

[


```

        {
            "name":
"get_weather",

            "description": "Get
weather info for
places",

            "parameters": {
                "type":
"dict",

                "required": [
                    "city"
                ],

                "properties": {
                    "city": {
                        "type":
"string",

                        "description": "The
name of the city to
get the weather for"
                    },
                    "metric":
{
                        "type":
"string",

                        "description": "The
metric for weather.
Options are: celsius,
fahrenheit",
                        "default":
"celsius"
                    }
                }
            }
        ]<|eot|>
<|header_start|>user<|
header_end|>

        What is
the weather in SF and
Seattle?<|eot|>

```

Model Response Format

```
<|header_start|>assistant<|header_end|>

[
  {
    "name":
    "get_weather",

    "parameters": {
      "city":
      "San Francisco",
    },
    {
      "name":
      "get_weather",

      "parameters": {
        "city":
        "Seattle",
      }
    }
  ]<|eot|>
```

Tool output sent back to the model

```
<|header_start|>ipython<|header_end|>

[
  {
    "response": "Sunny 75"
  },
  {
    "response": "Rainy 65"
  }
]<|eot|>
```

Final model response

```
<|header_start|>assistant<|header_end|>
```

```
San Francisco is sunny with a temperature of 75°F, while Seattle is rainy with a temperature of 65°F. <|eot|>
```

Was this
page
helpful?



Products

- API
- API waitlist
- Login

Models

- Llama 4
- Llama 3
- Download

News

- AI at Meta
- Blog
- Meta Newsroom

Documentation

- Overview
- Models
- How to guides
- Deployment guides
- Integration guides

Learn

- Cookbook
- Videos
- FAQ

AI research

- Overview

Community

- Case studies
- AI research community
- Llama Impact Grants
- Hugging Face
- GitHub

Llama Protections

- Overview
- Llama Defenders Program
- Developer Use Guide

Terms & policies

- Terms of Service
- Privacy Policy
- Cookies